Process Simulator: an open, browser-based BPMN simulation environment with statistically sound scenario comparison and simulation-based value stream mapping

Moritz Maier

Bern University of Applied Sciences (BFH), Bern, Switzerland

Abstract

What-if analysis with business process simulation is usually done in tools that compare a single simulated run per scenario, and that report a value stream map only for straight sequences of steps. We present Process Simulator, an open-source (Apache-2.0) discrete-event simulator for BPMN 2.0 that runs in the browser, as a Camunda Modeler plugin and as a command-line runner. It compares scenarios with replications, confidence intervals and common random numbers from per-source random streams, and derives value stream figures (lead time, process cycle efficiency, value-added and non-value-added time, rework and waiting) that also cover exclusive and parallel branches and loops. We verify the engine against exact queueing results (M/M/1, M/M/c, M/G/1, tandem and Jackson networks): all 13 of 13 confidence intervals cover the exact value, worst relative error 0.86 %. A cross-check with Prosimos shows no significant difference in 13 of 13 quantities; on a model discovered with Simod from the BPI Challenge 2012 log both engines agree within +0.6 % in mean cycle time. The value stream figures match closed-form results for sequences, choices, parallel splits and loops, where a linear map misstates process cycle efficiency. On a staffing problem, one run per scenario picks a wrong option in up to 45.1 % of the trials; ten paired replications reduce this to 0.5 %. All results can be recomputed with one command.

Email address: moritz.maier@bfh.ch (Moritz Maier)

Keywords: Business process simulation, BPMN, Discrete-event simulation, Common random numbers, Value stream mapping, Verification and validation, Open-source software

1. Introduction

A coffee-shop owner weighs three changes to her process: hiring a second barista, shifting the product mix to fifty-fifty, or both. She draws the process in BPMN, runs each option once in a simulation tool, and compares the reported mean cycle times. This is how wrong decisions are made: in the decision experiment of this paper, the option picked this way is wrong in up to 45.1 % of the trials (Section 4.5), although nothing about the model differs between the runs; only the random numbers do.

Business process simulation (BPS) estimates the effect of changes such as additional staff, shorter activities or different routing before they are implemented (Dumas et al., 2018; van der Aalst, 2015). Many BPM suites contain a simulator, and dedicated engines exist in research, notably BIMP (QBP) and Prosimos (López-Pintado & Dumas, 2022; López-Pintado, Dumas, & Berx, 2024), together with tools that discover simulation models from event logs (Camargo et al., 2020; Rozinat et al., 2009). Surveys of tool support report heterogeneous and often limited simulation capabilities (Laue & Müller, 2016; Pereira & Freitas, 2016; Rosenthal et al., 2021).

Two practical gaps motivated the tool presented here. First, what-if results are commonly read from a single run per scenario. The statistical remedies, independent replications, confidence intervals and common random numbers (CRN), are textbook simulation methodology (Kim & Nelson, 2001; Law, 2015; Nelson & Matejcik, 1995), but they are rarely part of the modelling workflow of BPMN tools. Second, value stream mapping (VSM) (Rother & Shook, 1999) is widely used to discuss lead time and waste, and simulation-based VSM is established in manufacturing (Lian & Van Landeghem, 2007; Solding & Gullander, 2009); process mining has been used to obtain value stream maps from logs (Knoll et al., 2019). A value stream map, however, assumes a linear sequence of steps, whereas BPMN models contain exclusive choices, parallel branches and loops.

This paper describes Process Simulator and, mainly, its verification and validation. We do not claim that BPMN simulation, simulation-based VSM or the combination of a model editor with a simulator are new. The contribution is an open, validated environment that integrates them:

1. modelling and simulation in one browser application without installation, and the same engine as a Camunda Modeler plugin and a command-line runner (Section 3);
2. scenario comparison with replications, Student-t confidence intervals and paired differences under CRN, where each source of randomness has its own stream so that changing one activity does not shift the random numbers of the others (Section 3.3);
3. value stream figures computed from the simulated time line of each case, which partition lead time also for choices, parallel branches and loops (Section 3.4);
4. an automated evaluation: exact queueing results, a cross-check with Prosimos, a model discovered from a public event log, closed-form VSM cases, decision quality of single runs versus replications, and run time (Section 4);
5. a reproducibility package under Apache-2.0 that recomputes every number in this paper with one command.

2. Related work

BPS tools and engines. BPS has a long tradition (van der Aalst, 2015); the BPSim standard defines simulation parameters for BPMN models (Laue & Müller, 2016). Pereira and Freitas (2016) and Rosenthal et al. (2021) compare simulation support in BPM tools and observe large differences in the supported parameters and outputs. Prosimos is an open-source Python engine with differentiated resources, i.e. individual calendars and performance per resource (López-Pintado & Dumas, 2022); it is the engine behind Simod, which discovers simulation models from event logs (Camargo et al., 2020; López-Pintado et al., 2024). Our engine is simpler in this respect: resources form roles with a shared calendar and a single duration distribution per activity.

Quality of simulation models. Discovered models are usually assessed by comparing simulated and real logs (Chapela-Campa et al., 2023, 2025) or by their usefulness for downstream tasks (Özdemir, Kirchdorfer, Amiri Elyasi, van der Aa, & Stuckenschmidt, 2025). Blümel, Kirchdorfer, Rehse, Kampik, and van der Aa (2026) show that data-preparation choices can change what-if conclusions on a real log. These works concern model discovery; we use a discovered model only to check that our engine reproduces a second engine on realistic input (Section 4.3).

Comparing alternatives. Selecting the best of several simulated systems is the subject of ranking and selection (Bechhofer, 1954; Kim & Nelson, 2001); CRN reduce the variance of differences between systems (Law, 2015; Nelson & Matejcik, 1995). Independent streams per source are a standard way to keep random numbers synchronised across scenarios (L'Ecuyer et al., 2002). Testbeds such as SimOpt collect simulation-optimisation problems (Eckman et al., 2023; Pasupathy & Henderson, 2006). We implement replications, confidence intervals and CRN; we do not implement a ranking-and-selection procedure.

Value stream mapping. Simulation-based VSM has been proposed for manufacturing systems to add dynamics to the static map (Lian & Van Landeghem, 2007; Solding & Gullander, 2009), and multidimensional process mining has been used to build value stream maps for internal logistics (Knoll et al., 2019). Our VSM view applies the same idea to BPMN models and defines the figures on the simulated case time line, so that they remain well defined for non-linear control flow.

3. The tool

3.1. Architecture

Figure 1 shows the application in simulate mode on the coffee-shop example shipped with the tool: the model carries annotations such as durations and branch probabilities, scenario tabs sit above the diagram, and the inspector on the right holds the demand, resource and activity parameters of the selected scenario.

Process Simulator is a TypeScript application. The modeller is bpmn-js; simulation parameters are stored inside the BPMN file as an extension element, so a model and its scenarios travel as one file. The discrete-event engine has no browser dependencies and runs in a Web Worker in the browser, in the Camunda Modeler plugin and in Node.js through the command-line runner `procsim`. The runner reads a BPMN file, runs one or all stored scenarios, or a scenario given as JSON, for a given number of replications, and writes one JSON line per run with the simulator version, truncated SHA-256 hashes of the model and the scenario, the seed, the replication index and the random-number mode. It also exports scenarios to Prosimos parameters and imports Prosimos parameters, e.g. from Simod.

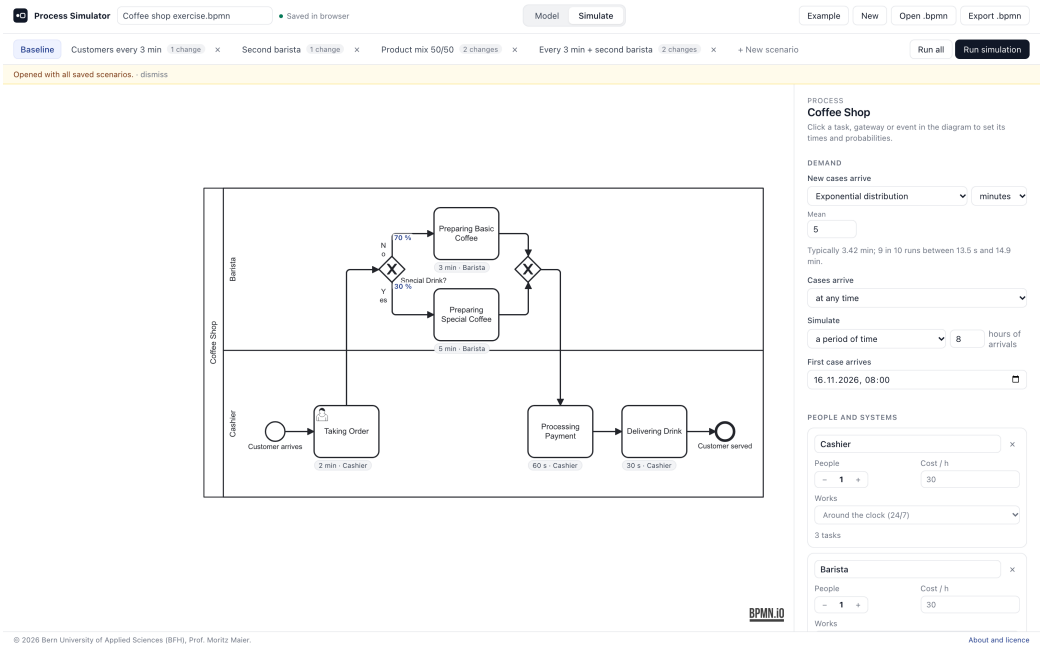


Figure 1: Process Simulator in the browser on the shipped coffee-shop example. Above the diagram, tabs switch between the baseline and the stored what-if scenarios; activity annotations show duration and role; the inspector edits the arrival process, resource pools and per-activity parameters of the selected scenario.

3.2. Simulation model

A scenario specifies the arrival process (inter-arrival distribution and optionally a calendar of the hours during which cases arrive), the number of cases or a time horizon, roles with a number of resources, an hourly rate and a weekly working-hours calendar, and for each activity a duration distribution (fixed, exponential, uniform, triangular, normal, lognormal or gamma), a role, an optional cost per run, revenue, queue limit, repetition probability and value class (value-added VA, business-value-added BVA or non-value-added NVA). Branch probabilities are set on the outgoing flows of exclusive and inclusive gateways. The engine executes BPMN token semantics for sequence flows, exclusive, inclusive, parallel, complex and event-based gateways, embedded sub-processes, multi-instance activities, intermediate timer and message events and boundary events. Activities wait in a first-come-first-served queue for a free resource of their role; work outside the calendar is paused and resumed at the next opening. Results include cycle, waiting

and processing times (mean and percentiles), throughput, work in progress, utilisation per role, cost and revenue, per-activity statistics and a simulated event log.

3.3. Statistical scenario comparison

Each scenario is simulated in R independent replications (default ten); replication i of every scenario uses the same seed s_i. In the default *keyed* mode, every random draw comes from a stream identified by its source (arrival, activity, gateway, boundary event or repetition) and the case number: the stream is a small-state generator (sfc32) seeded with a 32-bit hash of the replication seed, the source key and the case. A change to one activity therefore leaves the arrival times, branch decisions and durations of all other sources unchanged, which is what CRN require (Nelson & Matejcik, 1995). A legacy mode with a single stream is kept for comparison (Section 4.5). Warm-up is specified as simulated time; cases that arrive before the end of the warm-up are excluded from all statistics.

For each output measure the tool reports the mean over replications with a 95 % Student-t confidence interval. Scenarios are compared with the baseline by the paired differences $d_i = Y_i^{\text{scenario}} - Y_i^{\text{baseline}}$; the difference is reported with its confidence interval and marked as not significant when the interval contains zero. The same intervals appear in the app, in the PDF and Excel reports and in the JSON output.

3.4. Simulation-based value stream mapping

For every completed case the engine records the intervals in which an activity of the case was being processed, together with the value class of the activity and whether the execution was a repetition (rework). The lead time of the case, from arrival to completion, is then partitioned by sweeping its time line: at every instant the case is assigned to exactly one category, in the order of precedence VA, BVA, rework, NVA, waiting, where waiting means that no activity of the case is in process. With parallel branches, an instant in which a VA activity and an NVA activity run simultaneously counts as VA; the NVA work remains visible in the activity statistics but does not extend the lead time. By construction the categories add up to the lead time of each case. The value stream view reports the mean of each category, the process cycle efficiency PCE = VA/lead time and the per-activity contributions; it also shows the classical linear figure, the sum of the activity times along

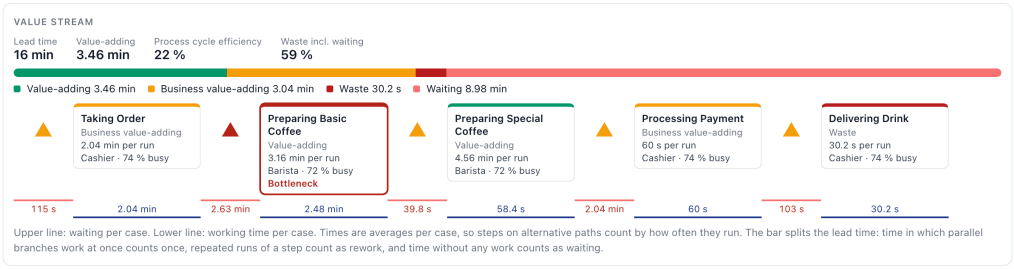


Figure 2: Simulation-based value stream map of the coffee-shop example. The bar partitions the mean lead time of 16 min into value-adding, business-value-adding, waste and waiting; the bottleneck activity is marked. Below, the upper line gives waiting and the lower line working time per activity. A linear map would add the durations of both coffee branches and report a higher PCE than the simulated 22 %.

the model, to make the difference visible. Figure 2 shows the view on the simulated coffee-shop baseline.

4. Evaluation

All experiments use the command-line runner of version 1.0.0 and are scripted in the paper repository; `make` recomputes data, tables and figures. Unless stated otherwise, 95 % confidence intervals are Student-t intervals over independent replications. The order of the experiments follows the question that opened this paper: Sections 4.1–4.4 establish that the engine and its value stream figures can be trusted, Section 4.5 measures how often the single-run shortcut fails, and Sections 4.6–4.7 place run time and feature coverage in context. Appendix Appendix A shows the test models.

4.1. V1: verification against exact queueing results

We built BPMN models whose steady-state behaviour is known analytically: M/M/1 at utilisation $\rho \in \{0.5, 0.8, 0.9\}$, M/M/3 and M/M/10, M/D/1, M/U/1 and M/G/1 with lognormal and gamma service (Pollaczek–Khinchine), a tandem of two M/M/1 stations and a Jackson network with feedback (Jackson, 1957). Each case ran 30 replications of 100 000 cases with a warm-up of 2000 mean inter-arrival times. Table 1 compares the simulated mean waiting time (or cycle time) with the exact value. All 13 of 13 intervals contain the exact value, and the largest relative error is 0.86 %. Little's law $L = \lambda W$ (Little, 1961), computed from the reported work in progress, throughput and cycle time, holds within 0.03 %.

Table 1: V1: simulated versus exact mean times in seconds.

Model	Quantity	Exact	Simulated (95 % CI)	Error	In CI
M/M/1, ρ=0.5	mean wait	30.0	30.0 ± 0.2	-0.14 %	yes
M/M/1, ρ=0.8	mean wait	192.0	191.4 ± 2.5	-0.33 %	yes
M/M/1, ρ=0.9	mean wait	486.0	483.8 ± 13.0	-0.46 %	yes
M/M/3, ρ=0.8	mean wait	155.3	154.9 ± 2.5	-0.24 %	yes
M/M/10, ρ=0.9	mean wait	361.1	360.7 ± 13.1	-0.11 %	yes
M/D/1, ρ=0.8	mean wait	96.0	95.9 ± 0.9	-0.10 %	yes
M/U/1, ρ=0.8	mean wait	104.0	104.1 ± 0.9	+0.09 %	yes
M/G/1 lognormal cv=1.5, ρ=0.8	mean wait	312.0	311.4 ± 7.6	-0.19 %	yes
M/G/1 gamma cv=0.5, ρ=0.8	mean wait	120.0	119.8 ± 1.1	-0.15 %	yes
Tandem M/M/1 to M/M/1	mean wait station 1	80.0	79.8 ± 0.6	-0.25 %	yes
Tandem M/M/1 to M/M/1	mean wait station 2	192.0	190.4 ± 2.7	-0.86 %	yes
Jackson feedback, p=0.25	mean wait per visit	144.0	143.9 ± 1.7	-0.06 %	yes
Jackson feedback, p=0.25	mean time in system	240.0	239.8 ± 2.3	-0.07 %	yes

4.2. V2: cross-check with Prosimos

Five models (tandem, M/M/3, exclusive choice, parallel split and rework loop) were exported with the built-in converter and simulated by Prosimos 2.1.0 and by Process Simulator, 20 replications of 50 000 cases each. The two engines use different random-number generators, so we compare means. Table 2 lists the relative difference with a Welch confidence interval. The intervals contain zero for 13 of 13 quantities and are within ±5.9 %. Eleven of the thirteen point estimates of Process Simulator are slightly higher (by 0.7–2.5 %), so a small systematic difference cannot be excluded at this sample size; it is of the order of the interval half-widths. This does not show that either engine is correct, but it shows that the two independent implementations agree on the semantics that both support. Features without a Prosimos counterpart, such as queue limits, revenue or boundary events, are listed by the exporter and were not used. One larger example model, the coffee shop shipped with the tool, could not be run by Prosimos (an internal error in its control-flow handling), which we report as a limitation of the cross-check.

4.3. V3: a model discovered from a public event log

To exercise the engine on realistic input we used the BPI Challenge 2012 log (van Dongen, 2012), restricted to the work items (activities starting with `W_`) with start and completion events, i.e. 9 657 cases. Simod 5.2.1 (Camargo et al., 2020) discovered a BPMN model with branching probabilities, resource profiles, calendars and duration distributions. Prosimos simulated this model directly; Process Simulator imported the Prosimos parameters.

Table 2: V2: Process Simulator versus Prosimos, mean times in seconds and relative difference with 95 % confidence interval.

Model	Quantity	This tool	Prosimos	Difference (95 % CI)
Sequence, two M/M/1 stations	mean cycle time	360.6	357.7	+0.8 % [-1.2, +2.8]
Sequence, two M/M/1 stations	mean wait t1	80.1	79.6	+0.7 % [-1.4, +2.7]
Sequence, two M/M/1 stations	mean wait t2	192.5	190.1	+1.3 % [-2.0, +4.5]
One activity, three servers	mean cycle time	301.3	298.7	+0.9 % [-1.3, +3.1]
One activity, three servers	mean wait t1	157.3	154.5	+1.8 % [-2.2, +5.9]
Exclusive choice 30/70, shared role	mean cycle time	289.2	292.1	-1.0 % [-3.7, +1.8]
Exclusive choice 30/70, shared role	mean wait a	236.5	239.7	-1.3 % [-4.7, +2.1]
Exclusive choice 30/70, shared role	mean wait b	237.4	240.2	-1.2 % [-4.5, +2.1]
Parallel split, two roles	mean cycle time	246.6	242.8	+1.6 % [-0.4, +3.5]
Parallel split, two roles	mean wait a	136.0	133.4	+1.9 % [-0.9, +4.7]
Parallel split, two roles	mean wait b	141.8	138.3	+2.5 % [-0.3, +5.3]
Rework loop p = 0.25	mean cycle time	243.8	239.8	+1.7 % [-0.6, +4.0]
Rework loop p = 0.25	mean wait work	146.8	143.8	+2.1 % [-0.7, +5.0]

Table 3: V3: BPI Challenge 2012 work items, log versus simulation of the discovered model, ten replications each (mean ± half-width of the 95 % confidence interval).

Measure	Log	Prosimos	This tool
mean cycle time (h)	252.02	3.40 ± 0.06	3.42 ± 0.04
work items per case	7.39	17.82 ± 0.06	17.88 ± 0.05

Because Process Simulator has one role and one duration per activity, the importer pools roles that share activities and merges the resource-specific durations of an activity into a gamma distribution with the same mean and variance; it reports 8 such approximations. Table 3 compares both engines with the log. The mean cycle times of the two engines differ by +0.6 %. Both engines also produce about 17.9 work items per case against 7.4 in the log, so the discovered control flow repeats activities far more often than the real process; the two engines agree with each other on this as well. The cycle times of both engines are far below the cycle time in the log, because the discovered model does not represent the time cases spend waiting outside the recorded work items; this is a property of the discovered model, not of either engine, and illustrates the threats discussed by Blümel et al. (2026).

4.4. V4: value stream figures for non-linear processes

Five models with closed-form lead time and partition were simulated with 30 replications: a sequence of three activities, an exclusive choice (30/70),

Table 4: V4: lead time (s), PCE and rework time (s): exact, simulated and from a linear value stream map.

Case	Lead time		PCE			Rework
	exact	tool	exact	tool	linear map	tool
Sequence	60.0	60.0	0.167	0.167	0.167	0.0
Exclusive choice (30/70)	58.0	57.9	0.517	0.515	0.714	0.0
Parallel split	50.0	50.0	1.000	1.000	0.625	0.0
Rework loop (p = 0.5)	40.0	39.9	0.500	0.502	1.000	19.9
Sequence with queues (M/M/1 tandem)	360.0	358.1	0.111	0.112	–	0.0

a parallel split, a rework loop with repetition probability 0.5 and a tandem with queues. Table 4 compares lead time and PCE with the exact values and with the PCE of a linear value stream map that adds the activity times along the model. The simulated figures agree with the exact ones within sampling error, and in every case the categories add up to the lead time (largest deviation 2×10^{-14} s). The linear map overstates PCE for the choice (because it adds both branches), understates it for the parallel split (because it adds parallel work) and ignores rework.

4.5. V5: single runs versus replications

We return to the question that opened this paper: which option is best, and how often does one run per option answer it wrongly? We took two decision problems: the coffee-shop example (four alternatives to the baseline) and an office process with a baseline and four staffing and duration options. The true ranking was obtained from 2000 replications per option. We then repeated, 2 000 times, the decision "pick the option with the lowest mean cycle time" with (a) one run per option, all with the same seed and one random stream, (b) one run per option with the same seed and per-source streams, (c) one run per option with independent seeds and (d) the mean of ten paired replications. The single-run and replicated decisions are drawn from the same replications that define the reference ranking; with 2000 replications per option the influence on the reference is small but not zero. Table 5 reports how often the true best option was chosen. On the office process, one run per option picks a wrong option in 45.1 % of the trials with independent seeds, 26.3 % with a shared seed and 19.8 % with per-source streams; ten paired replications reduce this to 0.5 %. The paired confidence intervals of the tool never declared a wrong direction ($\leq$0.0 %) and returned "not significant" in at most 4.2 % of the comparisons with the baseline. Table 6 shows why:

Table 5: V5: share of trials in which the truly best option is chosen.

Problem	One run per scenario			10 paired runs
	one stream, same seed	per-source streams	independent seeds	mean
Coffee shop	99.7 %	100.0 %	80.7 %	100.0 %
Office process	73.7 %	80.2 %	54.9 %	99.5 %

Table 6: V5: variance of the difference to the baseline in mean cycle time (s^2) for one run per option.

Problem	Option	True difference (s)	Variance of the difference		
			independent	one stream	per source
Coffee shop	Customers every 3 min	+946.3	315254	253735	241456
Coffee shop	Second barista	-153.6	27568	9892	10469
Coffee shop	Product mix 50/50	+182.1	102685	39434	30226
Coffee shop	Every 3 min + second barista	+162.5	27443	16634	17676
Office process	Second person at step 1	-179.5	18404	5501	3018
Office process	Step 2 10 % faster	-118.1	17619	4123	2129
Office process	Step 3 11 % faster	-59.3	21727	5081	266
Office process	Second person at step 3	-126.8	20840	5193	762

per-source streams reduce the variance of differences by up to a factor of 19 compared with a single stream. For the coffee-shop owner of the introduction this is the difference between acting on a coin flip and acting on evidence: ten paired replications answer the question correctly in all but 0.5 % of the trials.

4.6. V6: performance

Run time measures the implementation on a given machine, not the tool as such: it depends on the processor, the JavaScript engine and the load on the machine, whereas the simulated results do not. We therefore ran the same workloads on one machine (Intel Xeon Processor) with the command-line runner (Node.js v20.20.2) and with the engine in a Web Worker, as in the web application, in headless Chromium 136.0.7103.25 and Firefox 137.0. Each point is the median of 5 replications and times the simulation only, not loading the model; Firefox reports these times in whole milliseconds. For the same seeds all three runtimes produced identical results (0 differences in 30 comparisons). Figure 3 shows that run time grows approximately linearly with the number of cases; one million cases of a one-activity process take 10.1 s in Node.js (about 74 000 cases per second), 7.4 s in Chromium and 9.8 s in Firefox. Run time grows slightly faster than linearly with the number of activities or parallel branches per case, since every token event is a heap

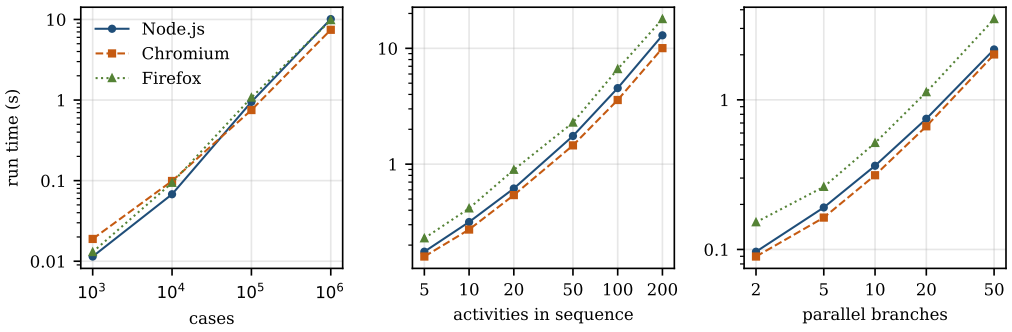


Figure 3: V6: run time by number of cases, sequence length and number of parallel branches, in Node.js and in a Web Worker in Chromium and Firefox on the same machine (logarithmic axes; median of 5 replications).

Table 7: V7: feature comparison based on public documentation.

Feature	Process Simulator	BIMP (QBP)	Prosimos / Simod
Runs in the browser, no installation	yes	yes (upload of a BPMN file)	no (Python)
Integrated BPMN editor	yes (bpmn-js, Camunda Modeler plugin)	no, external modeller	no, external modeller
Command-line / batch use	yes	not documented	yes
Discovery from event logs	no (imports Simod output)	no	yes (Simod) (Camargo et al., 2020)
Differentiated resources (individual calendars and durations)	no, roles	no	yes (López-Pintado & Dumas, 2022)
Replications with confidence intervals	yes	not documented	not documented
Common random numbers across scenarios	yes, per-source streams	not documented	not documented
Paired scenario comparison	yes	not documented	not documented
Value stream figures (VA/BVA/NVA, PCE)	yes, simulation-based	no	no
Licence	Apache-2.0	not documented	see repositories

operation on a larger event list and more per-activity statistics are kept. Across all workloads the browsers took between 0.7 and 1.7 times as long as Node.js. On other hardware the absolute times will differ, but the scaling is a property of the engine.

4.7. V7: comparison with other tools

Table 7 positions Process Simulator against two openly documented research tools that serve the same purpose, BIMP (the online front end of the QBP simulator) and Prosimos with Simod. The entries for the other tools are taken from their public documentation (BIMP web site, Prosimos and Simod repositories, consulted in November 2026) and from the publications cited; "not documented" means that we did not find the feature described, not that it is absent. Commercial suites with simulation modules were not included because their feature sets could not be checked from public sources in a comparable way; surveys of such tools are available (Pereira & Freitas, 2016; Rosenthal et al., 2021).

5. Discussion and limitations

The verification covers steady-state queueing behaviour, the semantics shared with Prosimos and the VSM partition; it does not cover every BPMN construct the engine executes. Event-based gateways, inclusive gateways, multi-instance activities and boundary events are covered by unit tests but not by an external reference. The resource model is coarser than that of Prosimos: there are no individual resource calendars, no resource-specific durations, no batching and no prioritisation, so models discovered by Simod have to be approximated (Section 4.3). The tool reports confidence intervals and paired differences but leaves the choice among several significant options to the user; ranking-and-selection procedures (Kim & Nelson, 2001) would be a natural extension. The decision experiment (V5) uses two problems and illustrates the effect of single runs; it is not a general estimate of error rates. The VSM categories depend on the value classes assigned by the modeller, and the precedence rule for parallel work is a design choice that we make explicit. No user study was conducted.

6. Conclusion

The question that opened this paper now has a defensible answer: a single run per scenario decides by chance, whereas replicated and paired comparisons decide by evidence. Process Simulator brings replicated, CRN-based scenario comparison and value stream figures for non-linear processes into a BPMN editor that runs in the browser and in the Camunda Modeler. The evaluation shows agreement with exact queueing results and with an independent engine, correct VSM figures for choices, parallel branches and loops, and a clear reduction of wrong decisions compared with single runs. The software, the experiments and the data are available under Apache-2.0.

Software and data availability

Software: `https://github.com/BFH-Process-Digitalization/process-simulator` (Apache-2.0); web application: `https://bfh-process-digitalization.github.io/process-simulator/`. The results reported here were produced with simulator release 1.0.0 (tag `v1.0.0`). Experiments, generated models, raw runs and this manuscript: `https://github.com/BFH-Process-Digitalization/process-simulator-paper`.

The BPI Challenge 2012 log is not redistributed; the scripts download it from 4TU.ResearchData (van Dongen, 2012).

CRediT authorship contribution statement

Moritz Maier: Conceptualization, Software, Validation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no competing interests.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used generative AI tools in order to draft and revise parts of the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

Bechhofer, R. E. (1954). A single-sample multiple decision procedure for ranking means of normal populations with known variances. *The Annals of Mathematical Statistics*, *25*(1), 16–39. https://doi.org/10.1214/aoms/1177728845

Blümel, R., Kirchdorfer, L., Rehse, J.-R., Kampik, T., & van der Aa, H. (2026). Glitches in the matrix: threats to the validity of data-driven business process simulation. *Process Science*, *3*, 19. https://doi.org/10.1007/s44311-026-00056-6

Camargo, M., Dumas, M., & González-Rojas, O. (2020). Automated discovery of business process simulation models from event logs. *Decision Support Systems*, *134*, 113284. https://doi.org/ 10.1016/j.dss.2020.113284

Chapela-Campa, D., Benchekroun, I., Baron, O., Dumas, M., Krass, D., & Senderovich, A. (2023). Can I trust my simulation model? Measuring the quality of business process simulation models. In *Business Process Management (BPM 2023)* (Vol. 14159, pp. 20–37). Springer. https://doi.org/ 10.1007/978-3-031-41620-0_2

Chapela-Campa, D., Benchekroun, I., Baron, O., Dumas, M., Krass, D., & Senderovich, A. (2025). A framework for measuring the quality of business process simulation models. *Information Systems*, *127*, 102447. https://doi.org/ 10.1016/j.is.2024.102447

Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018). *Fundamentals of business process management* (2nd ed.). Springer. https://doi.org/ 10.1007/978-3-662-56509-4

Eckman, D. J., Henderson, S. G., & Shashaani, S. (2023). SimOpt: A testbed for simulation-optimization experiments. *INFORMS Journal on Computing*, *35*(2), 495–508. https://doi.org/ 10.1287/ijoc.2023.1273

Jackson, J. R. (1957). Networks of waiting lines. *Operations Research*, *5*(4), 518–521. https://doi.org/ 10.1287/opre.5.4.518

Kim, S.-H., & Nelson, B. L. (2001). A fully sequential procedure for indifference-zone selection in simulation. *ACM Transactions on Modeling and Computer Simulation*, *11*(3), 251–273. https://doi.org/ 10.1145/502109.502111

Knoll, D., Reinhart, G., & Prüglmeier, M. (2019). Enabling value stream mapping for internal logistics using multidimensional process mining. *Expert Systems with Applications*, *124*, 130–142. https://doi.org/ 10 .1016/j.eswa.2019.01.026

Laue, R., & Müller, C. (2016). The business process simulation standard (BPSIM): chances and limits. In *Proceedings of the 30th European Conference on Modelling and Simulation (ECMS 2016)* (pp. 413–418). https://doi.org/ 10.7148/2016-0413

Law, A. M. (2015). *Simulation modeling and analysis* (5th ed.). McGraw-Hill.

L'Ecuyer, P., Simard, R., Chen, E. J., & Kelton, W. D. (2002). An object-oriented random-number package with many long streams and substreams. *Operations Research*, *50*(6), 1073–1075. https://doi.org/ 10.1287/opre.50.6.1073.358

Lian, Y.-H., & Van Landeghem, H. (2007). Analysing the effects of Lean manufacturing using a value stream mapping-based simulation generator. *International Journal of Production Research*, *45*(13), 3037–3058. https://doi.org/ 10.1080/00207540600791590

Little, J. D. C. (1961). A proof for the queuing formula: $L = \lambda W$. *Operations Research*, *9*(3), 383–387. https://doi.org/ 10.1287/opre.9.3.383

López-Pintado, O., & Dumas, M. (2022). Business process simulation with differentiated resources: does it make a difference? In *Business Pro-*

cess Management (BPM 2022) (Vol. 13420, pp. 361–378). Springer. https://doi.org/ 10.1007/978-3-031-16103-2_24

López-Pintado, O., Dumas, M., & Berx, J. (2024). Discovery, simulation, and optimization of business processes with differentiated resources. *Information Systems*, *120*, 102289. https://doi.org/ 10.1016/j.is.2023.102289

Nelson, B. L., & Matejcik, F. J. (1995). Using common random numbers for indifference-zone selection and multiple comparisons in simulation. *Management Science*, *41*(12), 1935–1945. https://doi.org/ 10.1287/mnsc.41.12.1935

Özdemir, K., Kirchdorfer, L., Amiri Elyasi, K., van der Aa, H., & Stuckenschmidt, H. (2025). Rethinking business process simulation: a utility-based evaluation framework. In *Process Mining Workshops* (pp. 128–144). Springer. https://doi.org/ 10.1007/978-3-032-02929-4_8

Pasupathy, R., & Henderson, S. G. (2006). A testbed of simulation-optimization problems. In *Proceedings of the 2006 Winter Simulation Conference* (pp. 255–263). https://doi.org/ 10.1109/wsc.2006.323081

Pereira, J. L., & Freitas, A. P. (2016). Simulation of BPMN process models: current BPM tools capabilities. In *New Advances in Information Systems and Technologies* (Vol. 444, pp. 557–566). Springer. https://doi.org/ 10.1007/978-3-319-31232-3_52

Rosenthal, K., Ternes, B., & Strecker, S. (2021). Business process simulation on procedural graphical process models. *Business & Information Systems Engineering*, *63*(5), 569–602. https://doi.org/ 10.1007/s12599-021-00690-3

Rother, M., & Shook, J. (1999). *Learning to see: Value stream mapping to add value and eliminate muda*. Lean Enterprise Institute.

Rozinat, A., Mans, R. S., Song, M., & van der Aalst, W. M. P. (2009). Discovering simulation models. *Information Systems*, *34*(3), 305–327. https://doi.org/ 10.1016/j.is.2008.09.002

Solding, P., & Gullander, P. (2009). Concepts for simulation based value stream mapping. In *Proceedings of the 2009 Winter Simulation Conference* (pp. 2231–2237). https://doi.org/ 10.1109/wsc.2009.5429185

van der Aalst, W. M. P. (2015). Business process simulation survival guide. In J. vom Brocke & M. Rosemann (Eds.), *Handbook on Business Process Management 1* (pp. 337–370). Springer. https://doi.org/ 10.1007/978-3-642-45100-3_15

van Dongen, B. (2012). *BPI challenge 2012.* 4TU.ResearchData,

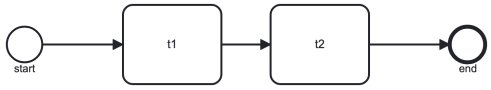
dataset. https://doi.org/ 10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f

Appendix A. Test models

The BPMN models used in V1, V2, V4 and V5 are generated by the model builder in `experiments/lib/bpmn.mjs` and stored in the `models/` directory of the paper repository; Figure A.1 shows them as rendered by bpmn-js. The BPI Challenge 2012 model of Section 4.3 is discovered by Simod 5.2.1 from the event log and regenerated by `make v3`; it is too large to reproduce legibly here.



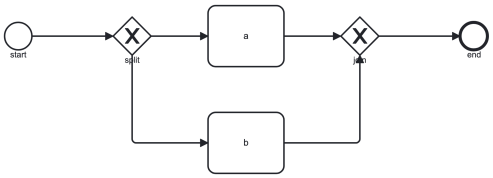
(a) single station (V1)



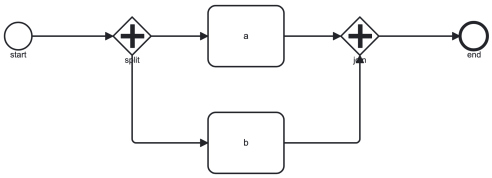
(b) tandem of two stations (V1, V2, V4)



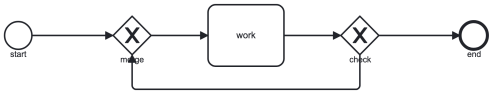
(c) three-step sequence (V4, V5 office process)



(d) exclusive choice (V2, V4)



(e) parallel split (V2, V4)



(f) rework loop, also the Jackson feedback network (V1, V2, V4)

Figure A.1: Generated BPMN test models used in the experiments; sources in `models/`.

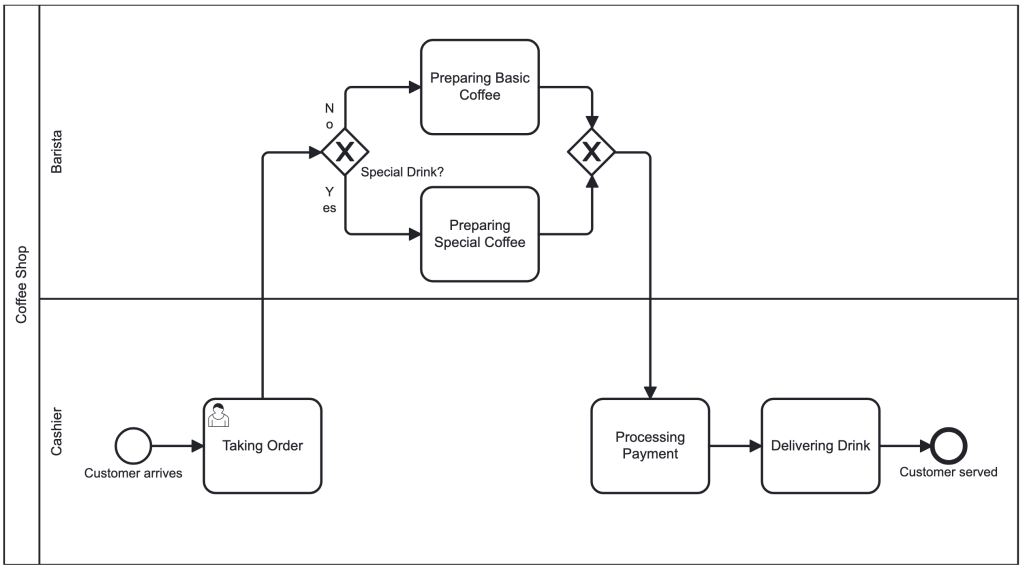


Figure A.2: The coffee-shop example shipped with the tool, used in the Prosimos cross-check (V2) and the decision experiment (V5).